



FOUR-COURSE SYSTEM

COURSE 3 - DATA ENGINEERING 2026

C3-040 Beginner-First Lesson Book

13 orchestration lessons rebuilt into the full M01 teacher loop.

Artifact	C3-040
Status	2026 - LEARNER EDITION
Phase	C3-03 Orchestration & Reliable Workflows 13 lessons 34-46 focused hours
Prerequisite	C3-02 Gate PASS; C2B Python/quality/ownership foundations assumed, orchestration taught here.
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House teaching rule: simple mental picture -> tiny example -> guided Airflow/tool workflow -> break/fix -> independent changed task -> validation -> production reasoning -> evidence. A green scheduler box alone never proves correct data.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

- | | |
|-----------------------------------|---|
| 01. Why it matters | 09. Why it worked |
| 02. What you learn | 10. Common beginner mistakes |
| 03. Prerequisite / starting state | 11. If stuck / recovery |
| 04. Picture it in your head | 12. Small independent task |
| 05. New words before use | 13. Validate |
| 06. Useful video/source | 14. Teach it back |
| 07. Follow with me once | 15. Protected review + changed retry |
| 08. Expected result | 16. Evidence/mastery + exact next route |

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Front matter	Page
This book is your teacher	2
Contents / Index	3
Module map + practice/review/Gate route	4
Phase-specific study notes	5
Lessons	Start
C3-03-L01 Workflow orchestration and DAG design	6
C3-03-L02 Airflow setup, standalone mode and project structure	9
C3-03-L03 DAG authoring with TaskFlow and tasks	12
C3-03-L04 Schedules, logical dates and data intervals	15
C3-03-L05 Parameters, variables, connections and secret boundaries	18
C3-03-L06 Retries, timeouts and failure callbacks	21
C3-03-L07 Catchup, backfills and controlled reprocessing	24
C3-03-L08 Sensors and data-aware dependencies	27
C3-03-L09 Idempotent task design and partial reruns	30
C3-03-L10 Pools, concurrency and resource limits	33
C3-03-L11 DAG testing and pre-deployment validation	36
C3-03-L12 Operational logs, metadata and troubleshooting	39
C3-03-L13 Deployment patterns, recovery runbooks and handoff	42
Mastery checklist + revision quick reference	45

Module map + protected learning route

Orchestration & Reliable Workflows

- 01 Prerequisite / prior-phase Gate
- 02 Lesson mental model + vocabulary
- 03 Follow guided example once
- 04 Matching independent practice
- 05 Save genuine attempt
- 06 Protected review unlocks
- 07 Close review + changed retry
- 08 Validate + explain back
- 09 Master lesson + save evidence
- 10 Mini-Lab / integrated drill
- 11 Protected Gate A
- 12 If needed: protected review -> targeted repair -> changed micro-proof
- 13 Explicitly assign fresh Gate B/C only after repair evidence
- 14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.



How to use this book

Each lesson is exactly three teaching pages: A = mental model/vocabulary/source, B = guided real tool example/result/debug, C = independent transfer/recovery/validation/mastery. External video is support; the book remains sufficient fallback.

C3-03-L01 - Workflow orchestration and DAG design

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

A scheduler can only recover cleanly when task boundaries correspond to meaningful, idempotent engineering units.

WHAT YOU LEARN

Model a data workflow as explicit tasks, dependencies, inputs, outputs and failure/recovery boundaries instead of one opaque script.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture a railway control board. Each station is one meaningful task, arrows show what must happen first, and a red signal stops unsafe downstream movement.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
orchestration	Coordinating when durable work starts, waits, retries and hands off.
DAG	Directed acyclic graph: tasks plus dependencies without cycles.
task	One schedulable unit of work.
dependency	Rule describing which work must finish or become ready first.
side effect	Persistent change such as writing a table/file.
failure boundary	Place where a failure can be diagnosed/retried without repeating unrelated work.

USEFUL SOURCE / VIDEO

RECOMMENDED - [Apache Airflow 3.3.1 - Architecture Overview](#)

Read Workloads + Control Flow. Purpose: see DAG/task/run architecture.



C3-03-L01 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Orchestration coordinates durable work; it is not the business logic itself. A DAG should reveal order, parallelism and blocking conditions. Task boundaries should align with independently testable outputs and useful retry/recovery points. Too many tiny tasks create operational noise; one giant task makes diagnosis and partial reruns impossible.

TINY WORKED EXAMPLE

Decompose a daily orders pipeline into resolve interval -> wait for input -> extract -> validate -> merge/load -> reconcile -> publish. Mark which failures should block downstream work and which tasks can be retried independently.

FOLLOW WITH ME ONCE

Draw the practice DAG and annotate: logical partition, persistent state, external dependency, quality gate, scarce resource, final publish.

EXPECTED RESULT

Every task has a single operational responsibility; publish is blocked by validation/reconciliation; no task depends on hidden local leftovers.

WHY IT WORKED

A scheduler can only recover cleanly when task boundaries correspond to meaningful, idempotent engineering units.

BREAK / FIX / DEBUG

If your design contains ``run_everything``, split first around observable outputs and failure boundaries. If it contains dozens of metadata-only tasks, combine steps that share one atomic outcome.



C3-03-L01 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Making one giant run_everything task.
- Splitting every trivial line into a task.
- Letting publish run without validation/reconciliation.
- Hiding dependencies in files/global state instead of DAG edges.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Redesign the same workflow when extraction comes from an API but loading still targets the same warehouse. Defend which task boundaries change and which do not.

VALIDATE

Every task has a single operational responsibility; publish is blocked by validation/reconciliation; no task depends on hidden local leftovers.

THEN SHOW THE PROFESSIONAL VERSION

You can design a fresh DAG with explicit dependency and recovery boundaries and defend the granularity of every task.

TEACH IT BACK

What belongs in Airflow, what belongs in ordinary Python/SQL, and why is that separation valuable during failure recovery?

PROTECTED REVIEW + CHANGED RETRY

Save P01 first. Only then open R01. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Saved DAG design + dependency reasoning + one failure-path walkthrough + changed transfer.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L02 - Airflow setup, standalone mode and project structure

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Reproducible setup prevents environment drift from masquerading as a DAG defect and keeps credentials out of Git.

WHAT YOU LEARN

Run a reproducible local Airflow 3.3.1 learning environment in WSL2 and keep durable DAG source separate from generated runtime state.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture a workshop. Your DAG files and requirements are the tools you own; the metadata database, logs and temporary credentials are sawdust created while working.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
WSL2	Linux environment used for Airflow on Windows.
AIRFLOW_HOME	Directory holding local Airflow runtime state/config by default.
standalone	Development command that starts a local Airflow environment.
constraints	Tested dependency-version set for reproducible Airflow installation.
metadata database	Airflow database storing scheduler/run/task state.
DAG folder	Directory Airflow scans for workflow source files.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow 3.3.1 - Installation from PyPI](#)

Use reproducible constraints install + pip check.

RECOMMENDED VIDEO - [Astronomer - Building ETL/ELT pipelines with Airflow 3](#)

Recorded 26 Mar 2026; 90 min. Focus: end-to-end workflow, quality, asset-aware scheduling/backfills. No verified chapter timestamp published on page.



C3-03-L02 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Airflow is designed for POSIX environments. On Windows, the course route is WSL2. `airflow standalone` is a development convenience that initializes local state and starts the required services together; it is not a production topology. Durable artifacts are source code, dependency pins, configuration expectations and runbooks. Local metadata DBs, logs, generated credentials and secrets are disposable/private state.

TINY WORKED EXAMPLE

Create `~/course3/orchestration/c3\_03`, a virtual environment, install Airflow 3.3.1 with its versioned constraints, run `airflow version`, start standalone, then verify the UI and `airflow dags list` without committing generated state.

FOLLOW WITH ME ONCE

Verify Python -> venv -> constrained Airflow install -> version -> standalone -> DAG discovery. Record which files are source-controlled versus runtime-generated.

```
cd ~/course3/orchestration/c3_03
source .venv/bin/activate
airflow version
airflow dags list
```

Use C3-039 for first install. Expected version: 3.3.1.

EXPECTED RESULT

Airflow reports 3.3.1; standalone starts; source project is separate from metadata/logs/secrets; no credential appears in Git-tracked files.

WHY IT WORKED

Reproducible setup prevents environment drift from masquerading as a DAG defect and keeps credentials out of Git.

BREAK / FIX / DEBUG

Dependency conflict -> recreate the venv using the exact Airflow 3.3.1 constraints URL. Windows path/permission oddity -> move the project into the WSL Linux filesystem. Resource pressure -> close Docker/Kafka/Spark while Airflow is running.



C3-03-L02 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Installing Airflow directly in Windows CMD/PowerShell.
- Using unconstrained pip install and ignoring dependency conflicts.
- Committing logs, metadata DB or generated credentials.
- Running Docker/Spark/Kafka alongside Airflow on an 8 GB laptop without need.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Delete only the disposable venv/runtime state, recreate the environment, and prove the project source still parses/discovers.

VALIDATE

Airflow reports 3.3.1; standalone starts; source project is separate from metadata/logs/secrets; no credential appears in Git-tracked files.

THEN SHOW THE PROFESSIONAL VERSION

You can recreate the local Airflow environment and explain every durable vs disposable project artifact.

TEACH IT BACK

Why is `airflow standalone` acceptable for this lab but not evidence of a production-ready deployment architecture?

PROTECTED REVIEW + CHANGED RETRY

Save P02 first. Only then open R02. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Version/startup/discovery evidence + clean project tree + secret hygiene check + recreated-environment proof.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L03 - DAG authoring with TaskFlow and tasks

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Thin DAGs are easier to test, upgrade, review and reuse; huge XCom payloads and hidden side effects couple task execution to scheduler metadata.

WHAT YOU LEARN

Author Airflow 3 DAGs through the public `airflow.sdk` interface, keep task logic thin, and exchange only compact metadata between tasks.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture a manager handing small job cards to workers. The manager coordinates who does what; the actual business work lives in reusable functions, not in the clipboard itself.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
TaskFlow	Decorator-based Airflow authoring style.
@dag	Decorator that defines a DAG factory.
@task	Decorator that turns a Python callable into an Airflow task.
airflow.sdk	Stable public authoring namespace in Airflow 3.
XCom	Small task-to-task metadata exchange mechanism.
import time	Work that happens merely because Python imports the DAG file; heavy I/O here is dangerous.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow 3 Task SDK / TaskFlow tutorial](#)

Use public airflow.sdk @dag/@task pattern and graph visualization.

RECOMMENDED VIDEO - [Astronomer - Writing functional DAGs with decorators](#)

Focus: decorator/TaskFlow DAG authoring and dependencies. Timestamp not verifiable from current landing page.



C3-03-L03 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Airflow 3 exposes stable DAG-authoring interfaces through `airflow.sdk`. TaskFlow turns Python callables into tasks and derives dependencies from outputs. XCom is appropriate for small metadata such as identifiers, row counts, paths and partition keys; large datasets belong in files, object storage, databases or tables. Keep pure transformation/load functions in normal modules so they can be unit-tested outside Airflow.

TINY WORKED EXAMPLE

Inspect `dags/c3\_03\_reliable\_orders\_TODO.py`: `@dag` defines scheduling/limits; `@task` wrappers call functions from `src/pipeline\_core.py`; task outputs pass partition/path/count metadata rather than rows/dataframes.

FOLLOW WITH ME ONCE

Wire `resolve_partition` -> `wait` -> `extract` -> `validate` -> `load` -> `reconcile` -> `publish`, then inspect the graph before running tasks.

```
from airflow.sdk import dag, task
```

```
@dag(schedule='@daily', catchup=False)
def reliable_orders():
    @task
    def resolve_partition(): ...
    partition = resolve_partition()
    # pass compact metadata, not the dataset itself
```

EXPECTED RESULT

DAG imports from `airflow.sdk`; import time has no I/O side effects; business logic remains testable outside Airflow; dependencies match the intended graph.

WHY IT WORKED

Thin DAGs are easier to test, upgrade, review and reuse; huge XCom payloads and hidden side effects couple task execution to scheduler metadata.

BREAK / FIX / DEBUG

If DAG import executes database/file work, move that work inside a task or pure module function. If XCom contains records/dataframes, persist them elsewhere and pass a reference.



C3-03-L03 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Putting all business logic in the DAG file.
- Passing a DataFrame/large dataset through XCom.
- Doing network/database I/O at DAG import time.
- Using private/deprecated imports instead of public Airflow 3 authoring interfaces.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Add a second compact metadata output (for example row_count) without passing the underlying dataset through XCom.

VALIDATE

DAG imports from `airflow.sdk`; import time has no I/O side effects; business logic remains testable outside Airflow; dependencies match the intended graph.

THEN SHOW THE PROFESSIONAL VERSION

You can author a fresh TaskFlow DAG using stable public interfaces and defend what should and should not travel through task outputs.

TEACH IT BACK

Why does a thin Airflow task wrapper improve both reliability and testability?

PROTECTED REVIEW + CHANGED RETRY

Save P03 first. Only then open R03. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Runnable DAG + graph evidence + pure-function test evidence + XCom/data-boundary explanation.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L04 - Schedules, logical dates and data intervals

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Interval correctness is the foundation of replay, backfill, reproducibility and auditability.

WHAT YOU LEARN

Make each scheduled run process the data interval it represents, including historical reruns, without substituting wall-clock 'today'.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture a calendar made of sealed daily boxes. A run belongs to one box. Reopening the September 2 box next week must still read and write September 2 data.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
schedule	Rule that creates DAG runs.
logical date	Identifier/time associated with the logical run.
data interval	Start/end of data that a run represents.
start date	Earliest schedule anchor, not simply first wall-clock execution.
catchup	Whether scheduler creates eligible missed historical runs.
wall clock	Actual current machine time; usually wrong for historical partition identity.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow 3.3.1 Scheduler / Dag Runs](#)

Focus: scheduler triggers after interval end; pair with data_interval_start/end.

RECOMMENDED VIDEO - [Astronomer - Scheduling in Airflow: Comprehensive Introduction](#)

Recorded 27 Jun 2023. Focus: scheduling terminology, time/event schedules, timetables. No verified chapter timestamps on page.



C3-03-L04 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

A scheduled DAG run represents a logical interval, and the run is typically created after that interval completes. Business partitions must be derived from Airflow's logical/data-interval context. ``datetime.now()`` is usually the wrong partition key because rerunning an old run tomorrow must still process its historical partition. Use timezone-aware start dates and deliberate catchup behavior.

TINY WORKED EXAMPLE

For a daily run representing 2026-09-02, derive ``orders_2026-09-02.csv`` from the interval/context. Compare with the incorrect ``date.today()`` approach and show how historical reruns would silently read a different file.

FOLLOW WITH ME ONCE

Use ``airflow dags next-execution``/UI interval information, derive the partition key, and verify input/output paths before running side-effect tasks.

```
# Correct mental model
partition = data_interval_start.date().isoformat()
# Wrong for historical replay:
# partition = date.today().isoformat()
```

EXPECTED RESULT

Historical run always maps to historical input/output; two intervals cannot overwrite each other; start date is timezone-aware; catchup choice is documented.

WHY IT WORKED

Interval correctness is the foundation of replay, backfill, reproducibility and auditability.

BREAK / FIX / DEBUG

Wrong partition -> inspect logical date/data interval and templates first; do not patch by changing the machine clock or hard-coding today's date.



C3-03-L04 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Using `datetime.now()` or `date.today()` for business partition paths.
- Thinking logical date equals actual start time.
- Changing catchup without understanding historical run creation.
- Letting different intervals overwrite the same output path.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Change the schedule from daily to a different interval and restate what one run represents before editing path logic.

VALIDATE

Historical run always maps to historical input/output; two intervals cannot overwrite each other; start date is timezone-aware; catchup choice is documented.

THEN SHOW THE PROFESSIONAL VERSION

You can make a fresh scheduled workflow interval-correct and prove it with historical rerun evidence.

TEACH IT BACK

Differentiate schedule time, logical date, data interval and actual wall-clock execution time.

PROTECTED REVIEW + CHANGED RETRY

Save P04 first. Only then open R04. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Two interval proofs + derived paths + catchup rationale + changed-schedule transfer.

MASTER only after the independent changed version passes validation and evidence is saved.



C3-03-L05 - Parameters, variables, connections and secret boundaries

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Reliable pipelines must be portable across environments without copying passwords into source or silently depending on global mutable state.

WHAT YOU LEARN

Separate per-run parameters, environment configuration, connection metadata and credentials so behavior is explicit without committing secrets.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture labeled drawers: Params = values chosen for this run; Variables = environment knobs; Connections = how to reach external systems; secrets = locked drawer contents never written in the DAG.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
Param	Validated runtime input to a DAG run.
Variable	Airflow-managed runtime configuration value.
Connection	Stored external-system endpoint/auth configuration identified by conn_id.
conn_id	Name used by tasks/hooks to retrieve a Connection.
secret	Credential/token/password that must not be committed to source.
configuration precedence	Which source wins when the same setting exists in multiple places.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow Params](#)

Focus: validated per-run configuration.

RECOMMENDED - [Airflow Connections](#)

Focus: conn_id and external-system auth boundary.



C3-03-L05 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Params are controlled run inputs. Variables are global runtime configuration and should not become a database for task-to-task data. Connections describe external systems; their credentials belong in environment/secrets mechanisms rather than DAG literals. Configuration precedence and ownership should be documented so a run can be reproduced without guessing which value won.

TINY WORKED EXAMPLE

Use a `min_rows`` Param, a non-secret environment/Variable for a default quality floor, and a connection ID for an incoming filesystem or warehouse. Keep only `.env.example`` / connection names in Git.

FOLLOW WITH ME ONCE

Classify eight values as Param / Variable-or-env / Connection / secret / code constant, then implement only the safe identifiers in source.

EXPECTED RESULT

No secret literal in tracked source; per-run values are explicit; connection identifiers are stable; config ownership/precedence is documented.

WHY IT WORKED

Reliable pipelines must be portable across environments without copying passwords into source or silently depending on global mutable state.

BREAK / FIX / DEBUG

Secret found in Git -> rotate it, remove it from current files and history per organizational procedure, then move to a secret-safe mechanism. Conflicting values -> document/verify precedence instead of guessing.



C3-03-L05 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Committing password/token literals.
- Using Variables as task-to-task data transport.
- Confusing per-run Params with deployment-wide configuration.
- Changing environment values without recording which value won.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Move the same DAG from local to another environment without changing DAG code; list which environment-owned values must be supplied.

VALIDATE

No secret literal in tracked source; per-run values are explicit; connection identifiers are stable; config ownership/precedence is documented.

THEN SHOW THE PROFESSIONAL VERSION

You can design configuration and secret boundaries for a fresh DAG and explain how the same code moves across environments.

TEACH IT BACK

Why is a connection ID safe to commit while the password behind it is not?

PROTECTED REVIEW + CHANGED RETRY

Save P05 first. Only then open R05. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Boundary table + safe config example + secret scan + changed-environment transfer.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L06 - Retries, timeouts and failure callbacks

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Blind retries can multiply side effects, hide permanent defects and delay incident detection.

WHAT YOU LEARN

Retry bounded transient failures, fail deterministic data defects immediately, bound hung work with timeouts and preserve useful failure context.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture a delivery driver. A temporary traffic jam deserves another attempt; an invalid address does not become valid by driving there five more times.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
transient failure	Temporary problem that may succeed later.
permanent failure	Deterministic problem needing repair, not repetition.
retry	New task attempt after failure.
backoff	Increasing delay between retries.
timeout	Maximum permitted wait/run duration.
callback	Operational action triggered by task/DAG state change.

USEFUL SOURCE / VIDEO

RECOMMENDED - [Airflow callbacks / task failure behavior](#)

Focus: failure callback as operational hook, not business retry logic.

RECOMMENDED VIDEO - [Astronomer - Debugging Airflow DAGs](#)

Focus: failure diagnosis/log evidence. Timestamp not verifiable from current page.



C3-03-L06 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Retries are for failures that may recover: transient network/service errors, short resource contention, etc. Invalid schema, duplicate-key violations and deterministic bad input usually should not be retried repeatedly. Retry delay/backoff and attempt limits protect dependencies; execution timeouts bound hangs. Failure callbacks/logs should add context while preserving the original exception.

TINY WORKED EXAMPLE

Use the packaged `fail_once()` guard: the first attempt raises a transient error and a retry succeeds. Contrast with a malformed timestamp fixture that must fail permanently without becoming green after repeated retries.

FOLLOW WITH ME ONCE

Classify five failures as transient/permanent/unknown, then attach an explicit retry/timeout policy and evidence requirement to each.

EXPECTED RESULT

Transient path shows failed attempt then success; permanent bad-data path stays failed; side effects remain idempotent; failure log retains run/task/partition context.

WHY IT WORKED

Blind retries can multiply side effects, hide permanent defects and delay incident detection.

BREAK / FIX / DEBUG

Retry storm -> reduce/disable retry for deterministic failure and investigate root cause. Duplicate side effect -> fix idempotence before relying on retry. Hanging task -> add timeout and external-call timeout.



C3-03-L06 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Retrying deterministic bad data repeatedly.
- Using many retries with no backoff/timeout.
- Assuming retry safety without idempotent side effects.
- Logging only "failed" without run/task/partition context.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Change the dependency from local file to rate-limited API and redesign retry/backoff without changing the business validation policy.

VALIDATE

Transient path shows failed attempt then success; permanent bad-data path stays failed; side effects remain idempotent; failure log retains run/task/partition context.

THEN SHOW THE PROFESSIONAL VERSION

You can create and prove a bounded failure policy for a fresh task.

TEACH IT BACK

Why is 'retries=5 everywhere' weaker engineering than explicit failure classification?

PROTECTED REVIEW + CHANGED RETRY

Save P06 first. Only then open R06. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Policy table + fail-once evidence + permanent-failure evidence + timeout rationale.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L07 - Catchup, backfills and controlled reprocessing

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Reprocessing is unavoidable in production; without idempotence and interval correctness it becomes data corruption with green scheduler boxes.

WHAT YOU LEARN

Reprocess historical intervals deliberately with explicit reprocessing behavior, bounded concurrency and partition-safe/idempotent outputs.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture missing pages in a ledger. Catchup can create missed scheduled pages automatically; a backfill is a deliberate request to reprocess chosen historical pages under explicit rules.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
catchup	Scheduler behavior for missed intervals.
backfill	Deliberate historical run creation/reprocessing.
reprocessing behavior	Rule deciding whether completed/failed historical runs may be recreated.
max active runs	Limit on concurrently active DAG runs.
historical partition	Past interval-specific input/output.
stale update	Older entity version that must not overwrite newer state.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow Backfill / CLI reference](#)

Focus: backfill create and reprocessing behavior.

RECOMMENDED VIDEO - [Astronomer - DAG Versioning and Backfills](#)

Recorded 15 May 2025. Focus topics: DAG versioning, supported backfills, quicker log access; no verified chapter timestamps on page.



C3-03-L07 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Catchup controls automatic creation of missed scheduled runs; backfill is a deliberate historical reprocessing action. Airflow 3.3.1 exposes `airflow backfill create` with `none`, `failed` and `completed` reprocessing behavior plus max-active-runs controls. A historical run is correct only if it processes the intended historical partition and does not corrupt newer state.

TINY WORKED EXAMPLE

Process 2026-09-01 through 2026-09-03, then backfill/reprocess an older partition after a newer version of O7002 already exists. The stale update must not overwrite the newer current row.

FOLLOW WITH ME ONCE

Plan backfill dates -> choose reprocess behavior -> bound active runs -> execute -> reconcile row keys/versions -> rerun one interval -> prove convergence.

```
airflow backfill create --dag-id <DAG_ID> --start-date <YYYY-MM-DD> --end-date <YYYY-MM-DD>
# choose reprocessing behavior deliberately; bound max active runs
```

EXPECTED RESULT

Correct historical partitions; no duplicate current keys; newest valid version retained; rerun materially unchanged; final totals reconcile.

WHY IT WORKED

Reprocessing is unavoidable in production; without idempotence and interval correctness it becomes data corruption with green scheduler boxes.

BREAK / FIX / DEBUG

Unexpected overwrite -> stop publish, inspect version/upsert rule, restore from preserved source/state, fix idempotence, then rerun only the affected interval. Excess pressure -> lower backfill active runs/pool capacity.



C3-03-L07 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Backfilling huge ranges with unbounded concurrency.
- Allowing an old partition to overwrite newer current state.
- Calling backfill and catchup the same thing.
- Reprocessing without before/after reconciliation.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Repeat with `failed`-only reprocessing and explain why it differs from deliberately recreating completed runs.

VALIDATE

Correct historical partitions; no duplicate current keys; newest valid version retained; rerun materially unchanged; final totals reconcile.

THEN SHOW THE PROFESSIONAL VERSION

You can safely plan, execute and validate a fresh historical reprocessing request.

TEACH IT BACK

Differentiate catchup from backfill and explain the meaning of none/failed/completed reprocessing behavior.

PROTECTED REVIEW + CHANGED RETRY

Save P07 first. Only then open R07. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Backfill plan + run evidence + stale-safe proof + rerun proof + reconciliation.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L08 - Sensors and data-aware dependencies

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Premature execution creates partial/incorrect results; wasteful polling reduces system capacity and can become an outage amplifier.

WHAT YOU LEARN

Wait for external readiness with an explicit sensor/dependency contract without burning workers in busy loops or letting downstream work run prematurely.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture a doorbell rather than someone holding the door handle all day. A sensor waits for readiness; efficient/deferrable waiting frees scarce worker capacity until the condition changes.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
sensor	Task specialized for waiting on an external condition.
poke	One check of a sensor condition.
deferrable	Can suspend and free worker capacity while waiting.
triggerer	Airflow component running lightweight asynchronous triggers.
readiness	Evidence that downstream work may safely start.
sensor timeout	Maximum total waiting period before failure.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow Deferrable Operators & Triggers](#)

Focus: release worker while waiting + triggerer behavior.

RECOMMENDED - [Astronomer - Sensors guide](#)

Focus: poke/reschedule/deferrable sensor choices and readiness.



C3-03-L08 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

A sensor models waiting for a condition: file arrival, partition readiness, external state or event. Waiting should have a timeout and an efficient execution mode. Deferrable/reschedule-style behavior can free worker capacity while waiting. The sensor proves readiness once; downstream tasks should not each implement their own polling loop.

TINY WORKED EXAMPLE

Use a FileSensor for the interval-specific incoming file, configured to wait efficiently. Show that downstream extract cannot begin before readiness and that a missing file reaches a clear timeout/failure rather than spinning forever.

FOLLOW WITH ME ONCE

Define readiness -> sensor condition -> timeout -> efficient waiting -> downstream dependency -> present/missing evidence.

EXPECTED RESULT

Downstream is blocked before readiness; present input proceeds; missing input fails within bound; waiting does not use a custom busy loop.

WHY IT WORKED

Premature execution creates partial/incorrect results; wasteful polling reduces system capacity and can become an outage amplifier.

BREAK / FIX / DEBUG

Sensor stuck -> inspect condition/path/connection/interval first. Worker starvation -> use appropriate deferrable/reschedule behavior rather than a tight loop or long blocking task.



C3-03-L08 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Busy-waiting while occupying a worker slot.
- Waiting forever with no timeout.
- Treating file existence as sufficient readiness without size/schema/completeness checks.
- Using a sensor for business processing instead of readiness.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Replace filesystem readiness with an upstream marker/table condition and preserve the same readiness/timeout contract.

VALIDATE

Downstream is blocked before readiness; present input proceeds; missing input fails within bound; waiting does not use a custom busy loop.

THEN SHOW THE PROFESSIONAL VERSION

You can model a fresh external readiness dependency and prove both success and bounded failure behavior.

TEACH IT BACK

Why is a sensor better than placing ``while not exists: pass`` inside a Python task?

PROTECTED REVIEW + CHANGED RETRY

Save P08 first. Only then open R08. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Readiness contract + success evidence + timeout evidence + changed-dependency transfer.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L09 - Idempotent task design and partial reruns

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Retries and operational recovery are normal. A non-idempotent task turns ordinary recovery into duplicate/corrupt data.

WHAT YOU LEARN

Make retries, cleared tasks and historical reruns converge to the same correct state for the same logical input.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture pressing an elevator button twice. The desired state is still one request. An idempotent task converges to the same correct business state even when retried or rerun.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
idempotent	Repeating the same logical operation converges to the same business state.
stable key	Persistent identity used to match the same entity/output.
upsert	Update matched identities and insert new ones.
atomic replace	Publish complete new output in one replace step instead of partially overwriting.
partial rerun	Re-executing only selected failed/downstream tasks.
convergence	Repeated execution reaches the same correct state.

USEFUL SOURCE / VIDEO

RECOMMENDED - [Astronomer - Rerunning DAGs](#)

Focus: clear/rerun mechanics; course controls decide idempotent side effects.



C3-03-L09 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Airflow guarantees orchestration attempts, not automatically safe side effects. Idempotent work uses stable business keys, interval-partitioned outputs, merge/upsert semantics, temporary writes plus atomic replace where appropriate, and explicit version rules. A task must not depend on undocumented leftovers from a prior failed attempt.

TINY WORKED EXAMPLE

Run the September 2 load twice against SQLite. The target remains one row per order_id, O7002 stays at its newest timestamp, and the interval summary file is atomically replaced rather than appended twice.

FOLLOW WITH ME ONCE

Stable partition -> stable business key -> deterministic merge -> atomic publish -> rerun -> compare state.

EXPECTED RESULT

Same logical input twice gives same business state; no duplicate keys; current version rule preserved; partial rerun touches only intended outputs.

WHY IT WORKED

Retries and operational recovery are normal. A non-idempotent task turns ordinary recovery into duplicate/corrupt data.

BREAK / FIX / DEBUG

Duplicate output -> stop using append-by-default; restore correct grain/version state, implement merge/replace semantics, then repeat same logical run until it converges.



C3-03-L09 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Appending on every retry.
- Using no stable business key/version rule.
- Clearing upstream tasks blindly after downstream-only failure.
- Calling code idempotent without same-run-twice evidence.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Change one source row to a stale earlier version and prove replay does not regress the current record.

VALIDATE

Same logical input twice gives same business state; no duplicate keys; current version rule preserved; partial rerun touches only intended outputs.

THEN SHOW THE PROFESSIONAL VERSION

You can make a fresh side-effecting task safe for retry and partial rerun and prove convergence.

TEACH IT BACK

Why must idempotence be designed in task code rather than assumed from Airflow retries?

PROTECTED REVIEW + CHANGED RETRY

Save P09 first. Only then open R09. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Two-run reconciliation + stale-version test + partial-rerun explanation.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L10 - Pools, concurrency and resource limits

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Resource guardrails prevent one DAG/backfill from degrading shared systems or crashing a small learning machine.

WHAT YOU LEARN

Protect scarce databases/APIs/learner hardware with explicit Airflow pool and concurrency limits instead of maximizing parallelism blindly.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture a one-lane bridge to a fragile database. A pool controls how many vehicles enter; more cars at once can create a traffic jam rather than more throughput.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
pool	Named Airflow capacity limit for tasks sharing a scarce resource.
pool slot	One unit of pool capacity.
concurrency	Amount of work allowed simultaneously.
max_active_runs	DAG-level cap on active runs.
queue	Tasks waiting for capacity.
throughput	Useful work completed per unit time, not simply number of parallel tasks.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow Pools](#)

Focus: pool slots and protecting scarce systems.

RECOMMENDED VIDEO - [Astronomer - Airflow production best practices](#)

Focus: resource/concurrency/operational practices. Timestamp not verifiable from current page.



C3-03-L10 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Pools limit how many tasks can compete for a scarce external resource. DAG active-run/task limits control pressure at other scopes. Priority only matters when work competes for constrained slots. Throughput is a systems property: more parallel tasks can reduce throughput if they saturate a database, API, memory or an 8 GB laptop.

TINY WORKED EXAMPLE

Assign database-writing tasks to `warehouse\_small`, set a deliberately small pool, and compare the graph/run behavior to unconstrained parallel work. Keep max_active_runs low for historical reprocessing.

FOLLOW WITH ME ONCE

Name scarce resource -> measure/assume safe slots -> create pool -> attach tasks -> bound active runs -> observe queue/running behavior.

EXPECTED RESULT

Tasks consuming the scarce resource share an explicit pool; limits are justified; a backfill cannot launch unbounded concurrent writes.

WHY IT WORKED

Resource guardrails prevent one DAG/backfill from degrading shared systems or crashing a small learning machine.

BREAK / FIX / DEBUG

Database/API pressure -> reduce concurrency before adding retries. Idle capacity -> measure before increasing. Resource contention -> separate pools by scarce dependency rather than by arbitrary DAG name.



C3-03-L10 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Maximizing parallelism because "faster is better."
- Creating pools by DAG name instead of scarce dependency.
- Ignoring backfill pressure on shared systems.
- Changing slot counts without evidence.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Move the same workflow to a slower API and redesign the pool/concurrency limits from the dependency's contract.

VALIDATE

Tasks consuming the scarce resource share an explicit pool; limits are justified; a backfill cannot launch unbounded concurrent writes.

THEN SHOW THE PROFESSIONAL VERSION

You can choose and defend resource limits for a fresh workflow.

TEACH IT BACK

Why can increasing concurrency make a data pipeline slower or less reliable?

PROTECTED REVIEW + CHANGED RETRY

Save P10 first. Only then open R10. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Pool config + resource rationale + observed scheduling evidence + changed-resource transfer.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L11 - DAG testing and pre-deployment validation

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

A scheduler is an expensive place to discover defects that deterministic local/CI checks could have caught first.

WHAT YOU LEARN

Catch syntax/import/dependency defects and business-logic failures before the scheduler discovers them in a scheduled production-like run.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture airport security layers. Unit tests check the bag contents, static/import checks verify the ticket, DAG discovery checks the passenger list, and an actual DAG test proves the journey can run.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
unit test	Fast test of pure business logic.
static check	Code/AST/lint check without scheduler execution.
DAG discovery	Airflow parsing and finding the DAG.
import error	Failure while Python imports/parses DAG source.
dags test	Airflow command for a focused logical-date DAG test.
CI	Automated pre-merge/pre-deploy checks.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow Best Practices](#)

Focus: DAG testing, import-time discipline and production-safe authoring.

REQUIRED - [Airflow CLI - dags test](#)

Focus: `airflow dags test`.



C3-03-L11 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Test pure business functions outside Airflow with pytest. DAG files should import quickly without network/database processing at import time. Pre-deployment checks include Python compile/static scan, unit tests, Airflow DAG discovery/import errors, graph/structure inspection and focused `airflow dags test` or task tests in the actual Airflow environment.

TINY WORKED EXAMPLE

Run packaged pytest tests, compile DAG/modules, execute the static AST audit, then in Airflow use `airflow dags list` and a focused DAG test. Introduce one dependency mistake and observe the pre-deploy check catch it before scheduling.

FOLLOW WITH ME ONCE

Compile -> pytest -> AST/static -> DAG discovery/import -> graph/test -> evidence bundle.

```
python -m pytest -q tests
python scripts/static_airflow_audit.py
airflow dags list
airflow dags test <DAG_ID> <LOGICAL_DATE>
```

EXPECTED RESULT

Pure tests pass; DAG compiles/imports; no import-time external work; dependency graph is intentional; runtime checks are clearly distinguished from static checks.

WHY IT WORKED

A scheduler is an expensive place to discover defects that deterministic local/CI checks could have caught first.

BREAK / FIX / DEBUG

DAG missing -> inspect import errors and dependencies. Unit failure -> fix business code, not orchestration. Slow import -> remove import-time I/O/large computation.



C3-03-L11 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Testing only the final scheduled run.
- Allowing I/O at DAG import time.
- Disabling assertions to make CI green.
- Equating Python compile with Airflow discovery/runtime correctness.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Add a fresh failing fixture to a pure function test and show that pre-deploy QA blocks the change before the DAG is scheduled.

VALIDATE

Pure tests pass; DAG compiles/imports; no import-time external work; dependency graph is intentional; runtime checks are clearly distinguished from static checks.

THEN SHOW THE PROFESSIONAL VERSION

You can design a layered pre-deployment test strategy and identify what each layer can and cannot prove.

TEACH IT BACK

Why do we test business logic outside Airflow even though the final workflow runs inside Airflow?

PROTECTED REVIEW + CHANGED RETRY

Save P11 first. Only then open R11. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Checklist + passing tests/static checks + learner Airflow discovery/test evidence + changed failing fixture proof.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L12 - Operational logs, metadata and troubleshooting

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

'Airflow is broken' cannot guide recovery. Precise evidence reduces mean time to diagnosis and prevents destructive reruns.

WHAT YOU LEARN

Diagnose scheduling, dependency, retry and data failures from specific run/task/interval evidence rather than vague symptoms.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture an aircraft flight recorder. Run ID, task ID, logical interval, state, logs and reconciliation controls let you reconstruct what happened instead of guessing from a red icon.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
DagRun	One execution of a DAG for a logical interval.
task instance	One task in one DAG run.
state	Current scheduler status such as queued/running/failed/success.
metadata	Structured scheduler/run/task information.
root cause	Underlying reason for failure rather than visible symptom.
reconciliation	Independent source-to-output correctness check after run/recovery.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow FAQ - scheduling/troubleshooting](#)

Focus: discovery, graph, scheduling checks and evidence.

RECOMMENDED VIDEO - [Astronomer - Debugging Airflow DAGs](#)

Focus: isolate failure from scheduler/run/task evidence. Timestamp not verifiable from current page.



C3-03-L12 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Troubleshooting starts by identifying the exact DAG run, task instance, logical interval and state. Separate parse/import failures, schedule/dependency blocks, sensor waits, task-runtime exceptions and data-quality failures. Useful logs record identifiers, counts, decisions and durations without secrets or raw sensitive records. Metadata and logs must support a timeline: what should have run, what did run, where did it stop, and what side effects occurred.

TINY WORKED EXAMPLE

Investigate four injected incidents: DAG import error, missing-file sensor timeout, transient fail-once retry, and permanent malformed timestamp. For each, identify the first decisive evidence and the safe next action.

FOLLOW WITH ME ONCE

Identify run -> identify task -> classify failure layer -> inspect relevant log/metadata -> inspect side-effect state -> choose minimal recovery -> reconcile.

EXPECTED RESULT

Diagnosis names the failure layer and evidence; recovery is minimal and idempotent; no secret values are copied into evidence.

WHY IT WORKED

'Airflow is broken' cannot guide recovery. Precise evidence reduces mean time to diagnosis and prevents destructive reruns.

BREAK / FIX / DEBUG

Start from state + log + interval + graph. Verify import errors before changing schedule; verify readiness before clearing sensors; verify target state before rerunning side-effect tasks.



C3-03-L12 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Reading only the last error line.
- Clearing the whole DAG before locating the failure boundary.
- Ignoring logical interval when diagnosing wrong data.
- Declaring recovery complete before reconciliation.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Diagnose a fresh 'queued but not running' case caused by a deliberately constrained pool rather than code failure.

VALIDATE

Diagnosis names the failure layer and evidence; recovery is minimal and idempotent; no secret values are copied into evidence.

THEN SHOW THE PROFESSIONAL VERSION

You can triage a fresh orchestration incident systematically and defend the chosen recovery action.

TEACH IT BACK

What evidence distinguishes 'not scheduled', 'waiting', 'failed during task code' and 'succeeded but produced wrong data'?

PROTECTED REVIEW + CHANGED RETRY

Save P12 first. Only then open R12. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

Four incident worksheets + one fresh constrained-pool diagnosis + recovery/reconciliation evidence.

MASTER only after the independent changed version passes validation and evidence is saved.

C3-03-L13 - Deployment patterns, recovery runbooks and handoff

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Pipelines outlive the engineer who wrote them. Reproducible deployment and recovery are part of correctness.

WHAT YOU LEARN

Package DAG source, dependencies, configuration expectations, tests, validation and recovery procedures so another engineer can operate and roll back/reprocess the workflow.

PREREQUISITE

C3-02 Gate PASS. Within this phase, previous C3-03 lesson mastery/remediation is expected. Bridge Python automation, quality, warehousing and professional ownership are assumed; Airflow-specific behavior is taught from zero.

PICTURE IT IN YOUR HEAD

Picture an emergency playbook beside a machine. The code explains how the machine is built; the runbook explains how another operator detects, diagnoses, recovers and proves it is healthy again.

NEW WORDS BEFORE WE USE THEM

Word	Meaning
deployment artifact	Versioned source/config needed to run workflow.
DAG bundle	Airflow mechanism for distributing/versioning DAG source.
runbook	Operator instructions for detect/diagnose/recover/validate.
rollback	Return to a known good code/config state.
reprocess	Run selected data interval again safely.
handoff	Transfer ownership with enough evidence for another engineer to operate the workflow.

USEFUL SOURCE / VIDEO

REQUIRED - [Airflow DAG Bundles](#)

Focus: versioned DAG distribution/deployment boundary.

RECOMMENDED - [Airflow Architecture Overview](#)

Focus: production components vs local standalone mental model.



C3-03-L13 - Follow with me once

Page B - tiny example, guided real tool workflow, expected result and break/fix

SIMPLE ENGINEERING IDEA

Production Airflow separates versioned DAG/source delivery from runtime-generated metadata/logs/secrets. Airflow 3 supports version-aware DAG execution concepts, but source control remains the durable change history. A handoff must state prerequisites, configuration, deployment verification, expected schedules/outputs, failure modes, rollback/reprocess steps, validation controls and ownership/escalation. A runbook is executable operational knowledge, not a vague README.

TINY WORKED EXAMPLE

Turn the practice project into a handoff packet: README, environment/setup, pre-deploy tests, deploy checklist, runbook, known failure modes, backfill command pattern, rollback/reprocess decision tree and evidence expectations.

FOLLOW WITH ME ONCE

Package source -> pin/describe runtime -> test -> deploy -> verify discovery/schedule -> document incidents -> define rollback/reprocess -> handoff ownership.

EXPECTED RESULT

No secrets/runtime state bundled; version/source identity recorded; deployment and recovery are reproducible; runbook includes validation after recovery.

WHY IT WORKED

Pipelines outlive the engineer who wrote them. Reproducible deployment and recovery are part of correctness.

BREAK / FIX / DEBUG

Unclear handoff -> add exact detection signal, first diagnostic checks, safe recovery scope, rollback/backfill conditions and post-recovery reconciliation. Do not say only 'rerun the DAG'.



C3-03-L13 - Now do it yourself

Page C - fresh transfer, mistakes, recovery, validation and mastery

COMMON BEGINNER MISTAKES

- Shipping source without dependency/config/runbook expectations.
- Treating local standalone layout as production architecture.
- No rollback/reprocess path.
- Handoff that requires the original author to remember undocumented steps.

STUCK? RECOVERY

1) State the intended logical interval and side effect. 2) Identify the exact task/config/state. 3) Reveal only the next structural step, not the full solution. 4) Open matching C3-042 only after saved genuine attempt. 5) Close review and solve a changed retry.

SMALL INDEPENDENT TASK

Assume the next release changes one task ID and one connection requirement. Write the migration/rollback note that prevents accidental historical confusion.

VALIDATE

No secrets/runtime state bundled; version/source identity recorded; deployment and recovery are reproducible; runbook includes validation after recovery.

THEN SHOW THE PROFESSIONAL VERSION

You can create and defend an operator-ready handoff for a fresh workflow.

TEACH IT BACK

Why is a DAG without a recovery procedure unfinished engineering when operational failure is plausible?

PROTECTED REVIEW + CHANGED RETRY

Save P13 first. Only then open R13. After review, close it and solve the changed check without reopening the answer. A review may repair understanding; it never erases the original attempt.

EVIDENCE / MASTERY

README + deploy handoff + runbook + rollback/reprocess path + second-person simulation evidence.

MASTER only after the independent changed version passes validation and evidence is saved.

Mastery checklist + revision quick reference

- ☐ Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- ☐ Protected review opened only after the attempt; changed retry solved after closing the review.
- ☐ Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- ☐ Mini-Lab / integrated drill completed with reproducible evidence.
- ☐ Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- ☐ No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- ☐ Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.